



PROGRAMAÇÃO I

Aula 05_2 Java (Semana 6)

Vinicius Bischoff

Quando você escreve um programa Java, é necessário ter um **ponto de entrada** onde o programa começa a ser executado. Esse **ponto de entrada** é o método "**public static void main(String[] args)**".

```
public class ExemploMain {
```

```
    public static void main(String[] args) {  
        System.out.println("Olá, mundo!");  
    }  
}
```

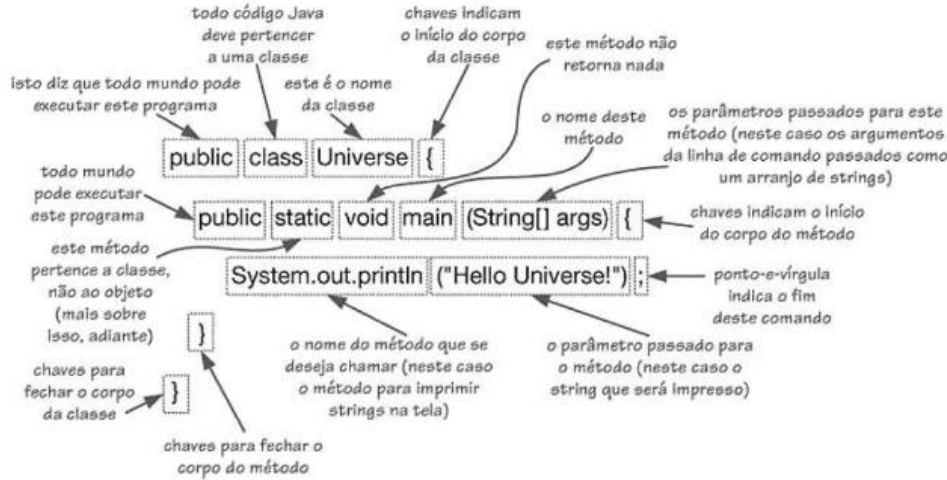
Vamos analisar cada parte dessa assinatura:

- "public": Esse é um modificador de acesso que indica que o método é público e pode ser acessado de qualquer lugar do código.
- "static": Esse modificador indica que o método é estático e pertence à classe em si, não a uma instância da classe. Isso significa que você não precisa criar um objeto da classe para chamar o método main.
- "void": Esse é o tipo de retorno do método, o que significa que ele não retorna nenhum valor.
- "main": Esse é o nome do método.
- "String[] args": Este é o parâmetro que o método recebe. Ele é um array de Strings que permite que você passe argumentos de linha de comando para o programa. Por exemplo, se você executar o programa a partir do terminal usando "java NomeDoPrograma argumento1 argumento2", esses argumentos serão passados para o parâmetro "args"

A combinação desses elementos forma o ponto de entrada para o programa Java.

Quando você executa um programa Java, o sistema inicia a execução a partir do método "main".

É dentro desse método que você escreve o código que será executado pelo programa.



A partir desse método, você pode chamar outros métodos e executar as tarefas necessárias para que seu programa funcione corretamente.



MinhaClasse.java

```
public class MinhaClasse {  
    public void meuMetodo() {  
        System.out.println("Olá do meu método!");  
    }  
}
```



ExemploMain.java

```
public class ExemploMain {  
  
    public static void main(String[] args) {  
        MinhaClasse minhaClasse = new MinhaClasse();  
        minhaClasse.meuMetodo();  
    }  
}
```

Para usar essa classe no método "main", podemos criar uma instância da classe "MinhaClasse" dentro do método "main" e chamar seu método "meuMetodo".

Criou-se uma instância da classe "MinhaClasse" com o nome "minhaClasse" e chamamos seu método "meuMetodo" usando o operador ".". Isso permite que você acesse e use os métodos e atributos de outras classes em seu programa Java.

Se as classes estiverem no mesmo pacote, não é necessário fazer um **import** para usá-las no método **main**. Isso ocorre porque as classes dentro do mesmo pacote têm acesso mútuo por padrão.

```
package meupacote;
```

Com o passar do tempo, você saberá o que cada palavra desta assinatura significa. Por enquanto, sabemos o que significam os termos **public** (indicando que o método é público), **void** (que não tem tipo de retorno) e **main** (que é o nome do método).

Também utilizamos o **main** para testar nossas classes e o comportamento dos nossos objetos. Então, é no método **main** que definimos os objetos das nossas classes interagem entre si e o que nosso programa deve fazer. *Mas, como criamos instâncias das nossas classes?* Utilizando a palavra reservada **new** e passando os parâmetros para o construtor, como fizemos no BlueJ.

Por exemplo, considere a classe Fruta, abaixo:

```
public class Fruta{  
    private String nome;  
    private double preco;
```

 **Fruta.java**

```
    public Fruta(String nome, double preco){  
        this.nome = nome;  
        this.preco = preco;  
    }
```

```
    public String getNome(){  
        return nome;  
    }
```

```
    public void setNome(String nome){  
        this.nome = nome;  
    }
```

```
    public double getPreco(){  
        return preco;  
    }
```

```
    public void setPreco(double preco){  
        this.preco = preco;
```

```
public class MainFruta {  
    public static void main(String[] args) {  
        // Criando uma instância de Fruta  
        Fruta f = new Fruta("Morango", 6.98);
```

```
        // Acessando e alterando o atributo preco  
        f.setPreco(5.76);
```

```
        // Acessando o atributo nome e armazenando em uma variável  
        String nomeFruta = f.getNome();
```

```
        // Chamando o método calculaPreco e armazenando o resultado em  
        // uma variável  
        double precoComImposto = f.calculaPreco(0.10);
```

```
        // Imprimindo os resultados
```

```
        System.out.println("Nome da fruta: " + nomeFruta);
```

```
        System.out.println("Preço da fruta: " + f.getPreco());
```

```
        System.out.println("Preço da fruta com imposto: " + precoComImposto);
```

```
    }  
}
```

 **MainFruta.java**

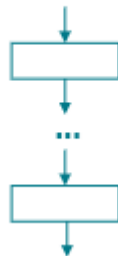
Instruções Condicionais

- Instruções podem ser executadas ou não, dependendo de certa condição. A lógica dos programas e pode ser feito com as instruções if, if-else e switch.
- Também é chamada de instrução de seleção
- Sintaxe do IF (observe que o primeiro não possui {})

```
if (condicao)  
    comando;
```

```
if (condicao) {  
    comando;  
}
```

Sequência



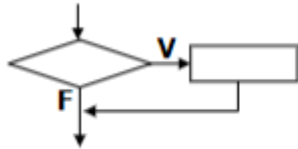
Dentro de um **retângulo** existe uma **ação** (ou seja, uma atribuição ou uma chamada de método).

Instruções de Repetição

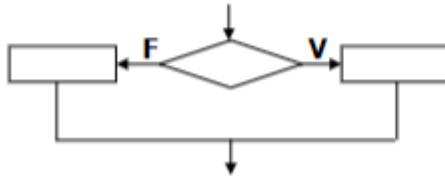
Dentro de um **losango** existe uma **comparação** (expressão do tipo **boolean**).

Estruturas de Seleção

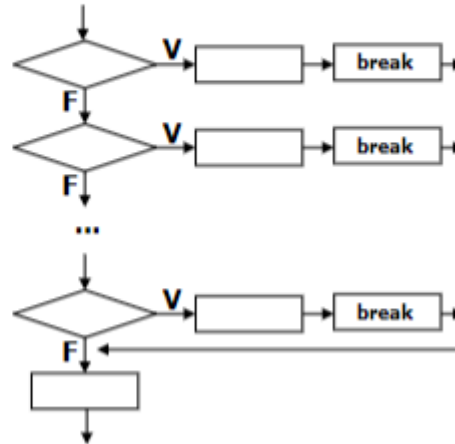
if



if-else



switch



- Onde: condição é qualquer valor ou expressão booleana
comando é qualquer instrução válida da linguagem
- Os comandos somente serão executados se a expressão de condição for verdadeira
- Caso o comando tenha apenas uma “linha”, então, não haverá necessidade de colocar bloco { ... }

Instruções Condicionais

- Exemplo com apenas um comando

```
// Importa a classe Scanner do pacote java.util
import java.util.Scanner;
// Cria a classe ExemploIfElse

public class ExemploIfElse {
    // Define o método main, que é o ponto de entrada do programa
    public static void main(String[] args) {

        // Cria um objeto Scanner para ler a entrada do usuário a partir do console
        Scanner scanner = new Scanner(System.in);
        // Pede ao usuário que digite sua idade e lê a entrada usando o objeto Scanner
        System.out.print("Digite sua idade: ");
        int idade = scanner.nextInt();
        // Testa se a idade é maior ou igual a 18 e imprime a mensagem correspondente
        if (idade >= 18) {
            System.out.println("Você é maior de idade.");
        } else {
            System.out.println("Você é menor de idade."); }
        // Fecha o objeto Scanner
        scanner.close();
    }
}
```



Instruções Condicionais

- Tarefa - Escreva um método situacaoIMC() na classe Pessoa, onde este receberá um valor real e retornará o status de “Problema”, caso esse número seja maior que 30.0. Caso contrário, “Normal”.

Instruções Condicionais

- IF...ELSE IF

Exemplo

```
if (idade < 10)
    faixaEtaria = "Criança";
else if (idade < 18)
    faixaEtaria = "Jovem";
else if (idade < 50)
    faixaEtaria = "Adulto";
else if (idade < 80)
    faixaEtaria = "Terceira idade";
else
    faixaEtaria = "Quarta idade";
```

Instruções Condicionais

- Tarefa - Altere o método situacaoIMC() e acrescente a seguinte informação:
- Abaixo de 18.5 (“Você está abaixo do peso ideal”)
- Entre 18.5 a 24.9 (“Parabéns – você está em seu peso normal”)
- Entre 25.0 a 29.9 (“Você está acima de seu peso sobrepeso”)
- Entre 30.0 a 34.9 (“Obesidade grau I”)
- Entre 35.0 a 39.9 (“Obesidade grau II”)
- A partir de 40.0 (“Obesidade grau III”)

Instruções de Repetição

- Existem também métodos específicos para comparação de valores em Java, como por exemplo, os métodos **equals()** e **compareTo()** para comparar objetos de classes diferentes.
- Em Java, a palavra-chave **final** é usada para indicar que um elemento não pode ser alterado. Ela pode ser aplicada a variáveis, métodos e classes.
- Variáveis: quando uma variável é declarada como **final**, ela não pode ser alterada após sua inicialização. Ou seja, seu valor é fixo durante toda a execução do programa.
- Métodos: quando um método é declarado como **final**, ele não pode ser sobrescrito por outras classes que herdam da classe que contém o método.
- Classes: quando uma classe é declarada como **final**, ela não pode ser herdada por outras classes.

Instruções de Repetição

Por exemplo, podemos usar 'final' para criar uma constante:

```
final double PI = 3.14159;
```

Ou para criar um método que não pode ser sobrescrito:

```
public final void imprimir() {  
    System.out.println("Imprimindo...");  
}
```

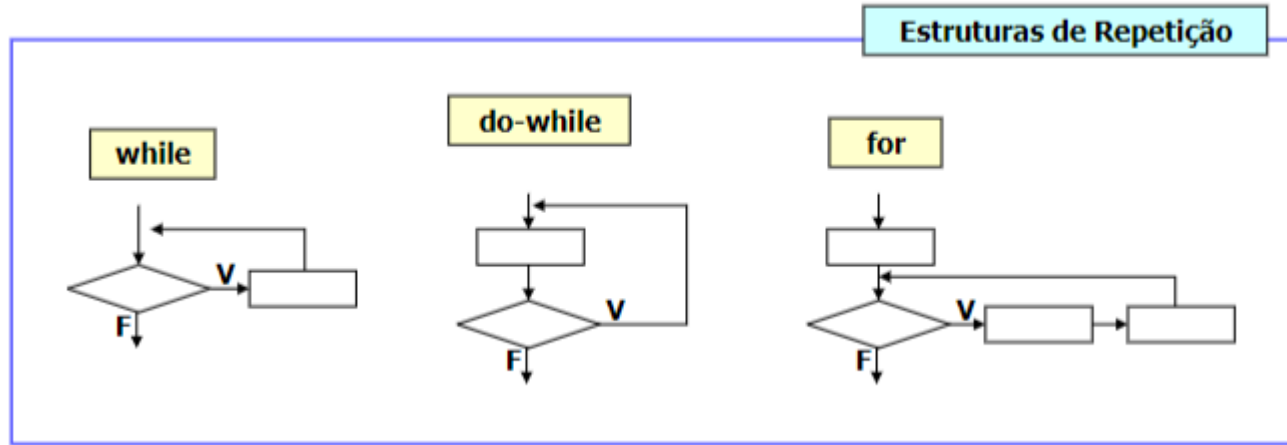
Ou para criar uma classe que não pode ser herdada:

```
public final class Pessoa {  
    // atributos, construtor, métodos  
}
```

Instruções de Repetição

- As instruções de repetição permitem fazer com que a execução de uma instrução (simples ou composta) seja repetida diversas vezes. Em Java existem três modalidades: **while**, **for** e **do**.

Instruções de Repetição



Observe que:

- while executa uma ação zero ou mais vezes.
- do-while executa uma ação uma ou mais vezes.
- for é uma estrutura de repetição que deve ser usada quando se sabe exatamente quantas vezes alguma ação deve ser repetida (repetição com base no valor de um contador)

Instruções de Repetição

- while

Permite repetir uma instrução ou um bloco, enquanto uma dada condição se mantiver verdadeira.

Sintaxe:

```
while (condição)  
    instrução; ou
```

```
while (condição) {  
    instrução1;  
    instrução2;  
}
```

Instruções de Repetição

- Inicialmente a condição é avaliada. Se der resultado true (verdadeiro), a instrução é executada e a condição volta a ser avaliada. Enquanto a condição for true (verdadeira), esse processo se repete.

Quando a condição for false, o controle saltará para a primeira instrução após o laço while.

Instruções de Repetição

- Inicialmente a condição é avaliada. Se der resultado true (verdadeiro), a instrução é executada e a condição volta a ser avaliada. Enquanto a condição for true (verdadeira), esse processo se repete.

Quando a condição for false, o controle saltará para a primeira instrução após o laço while.

Instruções de Repetição

```
public class ExemploWhile {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 5) {  
            System.out.println("O valor de i é: " + i);  
            i++;  
        }  
        System.out.println("Laço while encerrado.");  
    }  
}
```

```
public class ExemploDoWhile {  
    public static void main(String[] args) {  
        int i = 0;  
        do {  
            System.out.println("O valor de i é: " + i);  
            i++;  
        } while (i < 5);  
        System.out.println("Laço do-while encerrado.");  
    }  
}
```

Instruções de Repetição

- for

Permite repetir um ou vários comandos, dentro de um intervalo de valores, do início ao fim desse intervalo.

Sintaxe:

```
for(expressão inicial; condição; expressão incremento)  
    instrução;
```

ou

```
for(expressão inicial; condição; expressão incremento){  
    instrução 1;  
    instrução 2;  
}
```

Instruções de Repetição

- A instrução **for**

```
public class ExemploFor {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            System.out.println(i);  
        }  
        System.out.println("Laço for encerrado.");  
    }  
}
```

Instruções de Repetição

- A instrução **for** equivale a uma instrução **while** do tipo:

```
while (condição) {  
    instrução;  
    expressão incremento;  
}
```

```
public class ExemploWhile {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 10) {  
            System.out.println(i);  
            i++;  
        }  
        System.out.println("Laço while encerrado.");  
    }  
}
```

Escopo de variáveis

- Os campos de uma classe são variáveis de escopo global.
- Variáveis declaradas dentro de métodos são variáveis de escopo local (também conhecidas como variáveis automáticas ou de pilha).
- Variáveis locais devem obrigatoriamente ser inicializadas antes de serem usadas pela primeira vez no método.
- Parâmetros de métodos também são variáveis locais. Não é necessário inicializá-los pois eles são inicializados pelo código que faz a chamada ao método.
- As variáveis locais de um método são criadas cada vez que o método é chamado. Estas variáveis irão existir até que o método termine sua execução (quando, então, são destruídas). Portanto, o mesmo nome de variável pode ser utilizado em diferentes métodos.

Obrigado.

